Data Management (A.A. 2025/26) – 21/09/2026

Problem 1. Consider the two transactions $T_1 = r_1(A)\, r_1(B)\, w_1(B)\, w_1(A)$, and $T_2 = r_2(A)\, r_2(B)\, w_2(B)\, w_2(A)$, and tell how many conflict-serializable schedules on the set $\{T_1, T_2\}$ of transactions exist, motivating the answer in detail.

Solution. There are only 2 conflict-serializable (CSR) schedules: the two purely serial schedules $\langle T_1, T_2\rangle$ and $\langle T_2, T_1\rangle$. We can prove this statement in one of the following two ways.

By using the definition of CSR For a schedule S on T_1 and T_2 to be conflict-serializable, it must be conflict-equivalent to one of the two serial schedules $\langle T_1, T_2\rangle$ and $\langle T_2, T_1\rangle$.

Suppose that S is conflict-equivalent to $\langle T_1, T_2\rangle$.

Since $w_1(A)$ and $r_2(A)$ are conflicting operations, $w_1(A)$ must precede $r_2(A)$ in S.

However, $w_1(A)$ is the last operation of T_1, and $r_2(A)$ is the first operation of T_2. Therefore, every operation of T_1 must precede every operation of T_2, which means that S is the serial schedule $\langle T_1, T_2\rangle$.

By symmetry, the only schedule conflict-equivalent to $\langle T_2, T_1\rangle$ is $\langle T_2, T_1\rangle$.

Therefore, there are exactly two conflict-serializable schedules.

By using the precedence graph Consider any non-serial schedule S whose first operation belongs to T_1. Since the first operation of T_1 is $r_1(A)$, we have:

- $r_1(A)$ precedes $w_2(A)$, which creates the edge $T_1 \to T_2$ in the precedence graph.
- Since S is non-serial, some operation of T_2 must occur before the last operation of T_1, which is $w_1(A)$. Consequently, $r_2(A)$, the first operation of T_2, must precede $w_1(A)$. This creates the edge $T_2 \to T_1$.

Thus, the precedence graph contains a cycle $T_1 \to T_2 \to T_1$, and S is not conflict-serializable.

The case where the first operation belongs to T_2 is analogous.

Therefore, the only conflict-serializable schedules are the two serial schedules.

Problem 2. Provide an example of three transactions, which have the following properties:

(i) When being executed with a 2PL scheduler, deadlock may occur.
(ii) For each pair of the three transactions, no deadlock can occur in any execution of that pair with a 2PL scheduler.

Motivate your answer by providing the definition of 2PL scheduler and proving that the three proposed transactions satisfy both properties *(i)* and *(ii)*.

Solution. A *2PL SchedulerA Two-Phase Locking (2PL)* scheduler manages the concurrent execution of transactions by issuing and releasing shared (sl) and exclusive (xl) locks according to three fundamental rules:

- Well-Formed Transactions: Before reading a data item x, a transaction T_i must acquire a shared lock $sl_i(x)$ or an exclusive lock $xl_i(x)$. Before writing to x, it must acquire an exclusive lock $xl_i(x)$.
- Legal Locking Schedule: A lock request can be granted only if it is compatible with the locks currently held by other transactions on the same data item. In particular:
  - Multiple transactions may simultaneously hold shared locks on the same data item.
  - An exclusive lock cannot coexist with any lock held by another transaction on the same data item.
- Two-Phase Locking Rule: Once a transaction releases any lock ($u_i(x)$), it enters its shrinking phase and is not allowed to acquire any further locks.

A deadlock occurs in a 2PL scheduler when there exists a directed cycle in the Wait-For Graph (WFG), where nodes represent transactions and a directed edge $T_1 \to T_2$ indicates that T_1 is blocked waiting for a lock held by T_2.

Now let us consider the following three transactions:

- $T_1 = w_1(A)\, r_1(B)$
- $T_2 = w_2(B)\, r_2(C)$
- $T_3 = w_3(C)\, r_3(A)$

To show property *(i)*, consider an execution in which the first write operation of each transaction is performed before any transaction attempts its second operation. A 2PL scheduler would be proceed as follows:

1. T_1 requests and is granted the lock on A
2. T_2 requests and is granted the lock on B
3. T_3 requests and is granted the lock on C
4. T_1 requests the lock on $B \implies T_1$ blocks, waiting for T_2; this leads to an edge $T_1 \to T_2$ in WFG;
5. T_2 requests the lock on $C \implies T_2$ blocks, waiting for T_3; this leads to an edge $T_2 \to T_3$ in WFG;
6. T_3 requests the lock on $A \implies T_3$ blocks, waiting for T_1; this leads to an edge $T_3 \to T_1$ in WFG.

Clearly, the execution above leads to the cycle $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ in WFG.

To show property *(i)*, consider any pair of the three transactions.

- T_1 and T_2 access only one common data item, namely B.
- T_2 and T_3 access only one common data item, namely C.
- T_1 and T_3 access only one common data item, namely A.

In each pair, one transaction only reads the common data item, whereas the other writes it.

Therefore, in any execution of the pair, at most one transaction can be waiting for a lock held by the other: the transaction holding the lock on the common data item cannot be blocked by the other transaction, since no other data item is shared between them.

Consequently, the WFG of any execution of the pair cannot contain a cycle.

Thus, no deadlock can occur when only two of the three transactions are executed concurrently.

Problem 3. Let R be a relation stored in a heap file consisting of 10,000 pages. Assume that 20 buffer frames are available.

1. Sort R using external sorting, describe its execution, and compute its cost in terms of page accesses.
2. What is the minimum number of buffer frames needed to sort R with this algorithm in only two passes?
3. Assume now the same setting as in Point 1. However, due to system overload, after the initial runs have been written to secondary storage, the number of available buffer frames is reduced to one quarter of the original number (i.e., after Pass 0, only 5 buffer frames are available). Describe the execution of the external sorting algorithm in this case and compute its cost in terms of page accesses.

Solution.

1. With 20 buffer frames, in Pass 0 we can sort 20 pages at a time. Therefore, the number of initial sorted runs is
$$\left\lceil \frac{10000}{20} \right\rceil = 500.$$
In each subsequent pass, at most $20-1 = 19$ runs can be merged. The number of runs produced at each pass is therefore
$$500 \longrightarrow \left\lceil \frac{500}{19} \right\rceil = 27 \longrightarrow \left\lceil \frac{27}{19} \right\rceil = 2 \longrightarrow 1.$$
Thus, the algorithm requires 4 passes in total. Since at each pass the entire relation is read and written, the total cost is
$$4 \cdot 2 \cdot 10000 = \boxed{80000}$$
page accesses.
Note that this result can be obtained even by applying the formula $2 \cdot B \cdot \lceil \log_{F-1} B \rceil = 2 \cdot 10000 \cdot \lceil \log_{19} 10000 \rceil = 80000$

2. For a two-pass external sorting algorithm, all the runs produced in Pass 0 must be merged in a single subsequent pass. Thus, if F is the number of available buffer frames, we need
$$\left\lceil \frac{10000}{F} \right\rceil \leq F - 1.$$
With $F = 100$, 100 runs are produced, but only 99 can be merged. With $F = 101$, 100 runs are produced and 100 can be merged. Therefore, the minimum number of buffer frames is
$$\boxed{F = 101}.$$

3. Pass 0 is performed with the original 20 buffer frames and therefore produces 500 initial runs, as in Point 1. After Pass 0, only 5 buffer frames are available. Hence, at most $5 - 1 = 4$ runs can be merged at a time.
The sequence of runs is therefore
$$500 \longrightarrow \left\lceil \frac{500}{4} \right\rceil = 125 \longrightarrow \left\lceil \frac{125}{4} \right\rceil = 32 \longrightarrow \left\lceil \frac{32}{4} \right\rceil = 8 \longrightarrow \left\lceil \frac{8}{4} \right\rceil = 2 \longrightarrow 1.$$

Thus, the algorithm requires 6 passes in total. The total cost is

$$6 \cdot 2 \cdot 10000 = \boxed{120000}$$

page accesses.

Note that this result can be obtained even by applying the formula $2 \cdot B + 2 \cdot B \cdot \left\lceil \log_{(F \cdot 0{,}25)-1} \lceil B/F \rceil \right\rceil = 2 \cdot 10000 + 2 \cdot 10000 \cdot \lceil \log_4 500 \rceil = 2 \cdot 10000 + 2 \cdot 10000 \cdot 5 = 120000$

Problem 4. Let $R(A, B)$ and $S(B, C)$ be two relations with 500 and 20,000 pages, respectively. Assume that 50 buffer frames are available.

Consider the computation of the natural join $R \bowtie S$ on the common attribute B.

Discuss the possible strategies for computing the join, considering two-pass and multi-pass hash-based and sort-based algorithms. For each of the two families of algorithms (hash-based and sort-based), identify the most efficient applicable strategy and compute its cost in terms of page accesses. Clearly state any assumptions on the data that are required for the proposed algorithms to be applicable. Assume that both relations are initially stored in heap files and that no indexes are available.

Solution.

Hash-based approach.

A two-pass hash-based join can be used under the assumption that a hash function on the join attribute B distributes the tuples approximately uniformly among the buckets.

In the first pass, both relations are partitioned into $M - 1 = 49$ buckets using the same hash function on the join attribute B.

For the smaller relation R, the expected size of each bucket is

$$\left\lceil \frac{B(R)}{M-1} \right\rceil = \left\lceil \frac{500}{49} \right\rceil = 11 \text{ pages.}$$

Thus, under the uniformity assumption, each bucket of R fits in memory during the second pass. For each pair of corresponding buckets R_i and S_i, R_i is loaded into memory and S_i is scanned to compute the join.

The first pass reads and writes both relations, whereas the second pass reads both relations once. Therefore, ignoring as usual the cost of writing the result, the total cost is

$$2(B(R) + B(S)) + (B(R) + B(S)) = 3(B(R) + B(S)).$$

Hence,

$$3(500 + 20000) = \boxed{61500}$$

page accesses.

Notice that without the assumption on the distribution of the tuples, some bucket of R could be too large to fit in memory, and therefore the two-pass algorithm would not be guaranteed to work. Moreover, without further assumptions on the data distribution, a multi-pass hash-based strategy would not be guaranteed to solve the problem either, since repartitioning may still produce buckets that are too large to fit in memory. Under the uniformity assumption made above, however, a two-pass hash-based join is already possible.

Sort-based approach.

We first consider whether a two-pass sort-based join can be used.

Simple two-pass sort-based join.

In Pass 0, sorted runs of at most $M = 50$ pages are created. Thus, for R we obtain

$$\left\lceil \frac{500}{50} \right\rceil = 10$$

initial runs, whereas for S we obtain

$$\left\lceil \frac{20000}{50} \right\rceil = 400$$

initial runs.

A merge pass can merge at most $M - 1 = 49$ runs. Therefore, while the 10 runs of R can be merged in one pass, the 400 runs of S cannot. The corresponding condition for a relation with B pages is

$$\left\lceil \frac{B}{M} \right\rceil \leq M - 1.$$

For S,

$$\left\lceil \frac{20000}{50} \right\rceil = 400 > 49.$$

Equivalently, neglecting rounding effects, a sufficient condition for the simple two-pass sort-based join can be expressed as

$$\sqrt{\max(B(R), B(S))} + 1 \leq M.$$

In the present case,

$$\sqrt{20000} + 1 \approx 142.42 > 50.$$

Consequently, a simple two-pass sort-based join cannot be used.

Two-pass sort-merge join.

We can also consider a two-pass sort-merge join. After the initial sorted runs have been created, the second pass performs the merge of the runs of both relations and the join simultaneously.

For this algorithm to be applicable, one buffer frame is needed for each run of both relations, in addition to one frame for the output. Therefore, the total number of runs of the two relations must not exceed $M - 1$. The corresponding condition is

$$\left\lceil \frac{B(R)}{M} \right\rceil + \left\lceil \frac{B(S)}{M} \right\rceil \leq M - 1.$$

In the present case,

$$\left\lceil \frac{500}{50} \right\rceil + \left\lceil \frac{20000}{50} \right\rceil = 10 + 400 = 410 > 49.$$

Therefore, the available buffer space is not sufficient to perform the sort-merge join in two passes.

Equivalently, using $\lceil x \rceil \leq x + 1$, a sufficient condition for the algorithm to be applicable can be expressed as

$$\frac{B(R)}{M} + 1 + \frac{B(S)}{M} + 1 \leq M - 1.$$

That is,

$$\sqrt{B(R) + B(S)} + 3 \leq M.$$

In the present case,

$$\sqrt{500 + 20000} + 3 = \sqrt{20500} + 3 \approx 146.18 > 50.$$

Consequently, a two-pass sort-merge join cannot be used either.

Multi-pass sort-based join.

Since neither of the two-pass sort-based strategies is applicable, we consider a multi-pass sort-based join.

After Pass 0, R consists of 10 sorted runs and S of 400 sorted runs. The runs of R need not be merged further at this stage.

For S, an additional merge pass can merge at most $M - 1 = 49$ runs at a time and therefore produces

$$\left\lceil \frac{400}{49} \right\rceil = 9$$

sorted runs.

At this point, there are 10 runs for R and 9 runs for S. Thus,

$$10 + 9 = 19 \leq M - 1 = 49,$$

so all these runs can participate in a final merge in which the join is computed.

The cost is therefore:

- Pass 0 on both relations:
$$2(B(R) + B(S));$$
- one additional merge pass on S:
$$2B(S);$$
- final merge and join:
$$B(R) + B(S).$$

Hence, ignoring the cost of writing the result,

$$2(B(R) + B(S)) + 2B(S) + (B(R) + B(S)),$$

which gives

$$3B(R) + 5B(S) = 3(500) + 5(20000) = \boxed{101500}$$

page accesses.

This final cost assumes that, for each join-key value, all tuples having that value from at least one of the two relations can be kept in memory while the corresponding tuples from the other relation are scanned. Under this assumption, each input page needs to be read only once during the merge phase. Otherwise, additional page accesses may be required to process groups of tuples with the same join-key value, for instance by using a nested-loop strategy for such groups.

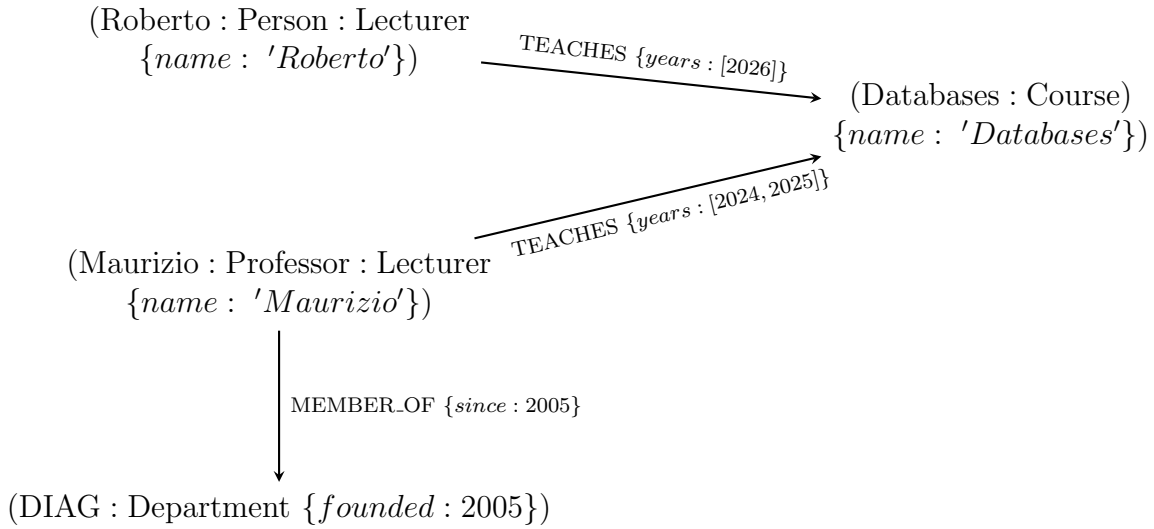
Problem 5. (only for students who opted for **option 1**, i.e., who do **not** do the project)

1. Create a property-graph database representing the following information (you may draw a labeled graph with properties specified between braces, as in the graphical representation of Neo4j graphs).
   - A person whose name is Roberto teaches a course called Databases in 2026.
   - A professor whose name is Maurizio teaches a course called Databases in 2024 and 2025.
   - Maurizio has been a member of DIAG since 2005.
   - DIAG is a department, founded in 2005.
   - Maurizio and Roberto are Lecturers.
2. Write an RDF dataset G to express the information described at Point 1, but replace the last point with "everyone who teaches is a Lecturer". Use the RDF/RDFS vocabulary where appropriate; otherwise, use any URI you prefer (assuming an empty prefix).
3. Write the SPARQL query Q that retrieves all Lecturers.
4. Provide the answers to Q when it is (a) evaluated over G interpreted as a graph database; and (b) computed over G interpreted as an RDFS Knowledge Graph (i.e., considering also the facts logically implied by G).

Justify your answers.

Solution.

1. One possible property graph is the following:



2. One possible RDF dataset G is:

```
:Roberto   rdf:type       :Person .
:Roberto   :name          "Roberto" .
:Maurizio  rdf:type       :Professor .
:Maurizio  :name          "Maurizio" .

:Databases rdf:type       :Course .
:Databases :name          "Databases" .

:Roberto   :teaches   :Databases2026 .
```

```
:Maurizio  :teaches   :Databases2024 .
:Maurizio  :teaches   :Databases2025 .

:Databases2024 . :edition_of :Databases .
:Databases2025 . :edition_of :Databases .
:Databases2026 . :edition_of :Databases .

:Databases2024 :year "2024" .
:Databases2025 :year "2025" .
:Databases2026 :year "2026" .

:Maurizio :has_membership _:x .
_:x :refers_to_department :DIAG .
_:x :since "2005" .

:DIAG rdf:type :Department .
:DIAG :founded "2005" .

:teaches   rdfs:domain    :Lecturer .
```

The last triple expresses that everyone who teaches is a Lecturer.

3. The SPARQL query Q is:

```
SELECT ?x
WHERE {
    ?x rdf:type :Lecturer .
}
```

4. If Q is evaluated over G interpreted simply as a graph database, the answer is empty:

$$Q(G) = \emptyset.$$

Indeed, G contains no explicit triple of the form

$$x \texttt{ rdf:type :Lecturer}.$$

If G is instead interpreted as an RDFS Knowledge Graph, from

```
:Roberto :teaches :Databases2026 .
:Maurizio :teaches :Databases2025 .
:teaches rdfs:domain :Lecturer .
```

we can infer that both Roberto and Maurizio are Lecturers. Therefore,

$$Q(G) = \{\texttt{:Roberto}, \texttt{:Maurizio}\}.$$